

CSC270 · DATABASE SYSTEMS · LECTURE 26

Beyond the Relational Model

Introduction to NoSQL Databases

CSC270 · Database Systems — COMSATS University Islamabad, Lahore Campus

Agenda

- 1 Where relational databases struggle at web scale
- 2 The shared characteristics that define “NoSQL”
- 3 Relational vs. NoSQL, dimension by dimension
- 4 The four major NoSQL families, with real products
- 5 Advantages traded for, and the limitations that come with them
- 6 Realistic use cases for reaching for NoSQL
- 7 The CAP theorem — why you can't have it all

Recap: Where We Left Off

- Relational design works well for normalized data on one strong server.
- At web scale it strains: vertical scaling has a ceiling, joins get expensive, schemas resist change.
- NoSQL trades strict schema and strong consistency for horizontal scale and flexibility.
- Four families — document, key-value, column-family, graph — each tuned to one access pattern.
- Next: how one document database, MongoDB, works in practice.

A Social-Media Startup, Three Stages of Growth

● 1,000 users

A single relational server handles everything comfortably — posts, comments, likes, all cleanly normalized.

● 10 million users

Writes alone start to outpace one server's disk and CPU. Buying a bigger server (vertical scaling) has a hard ceiling.

● 500 million users

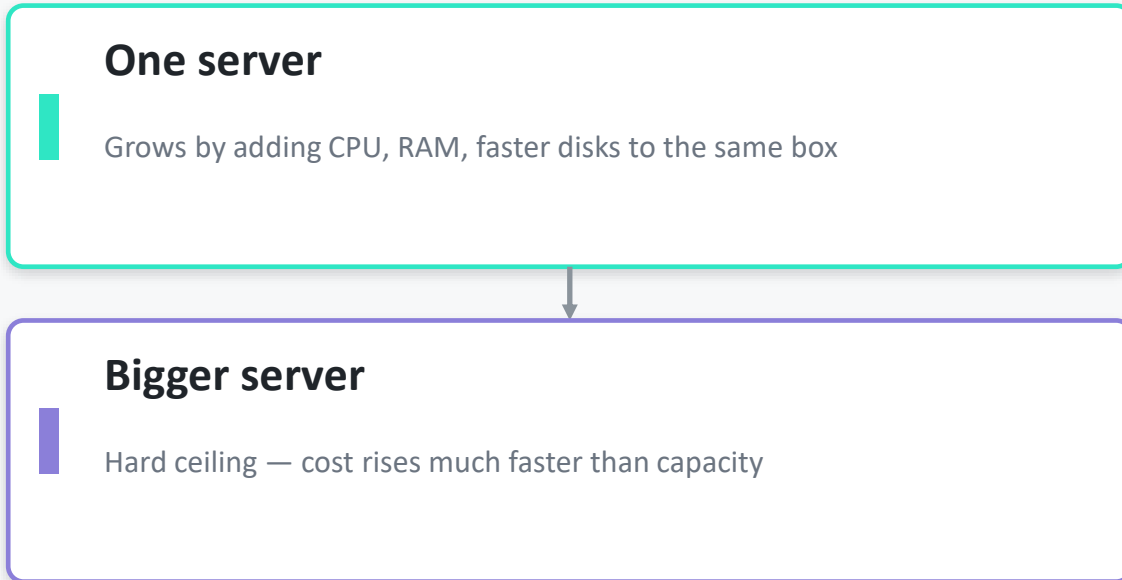
Friend-of-a-friend joins get expensive; every new post “type” means another schema migration nobody can afford to lock for long.

■ NOT “WRONG,” JUST OPTIMIZED DIFFERENTLY

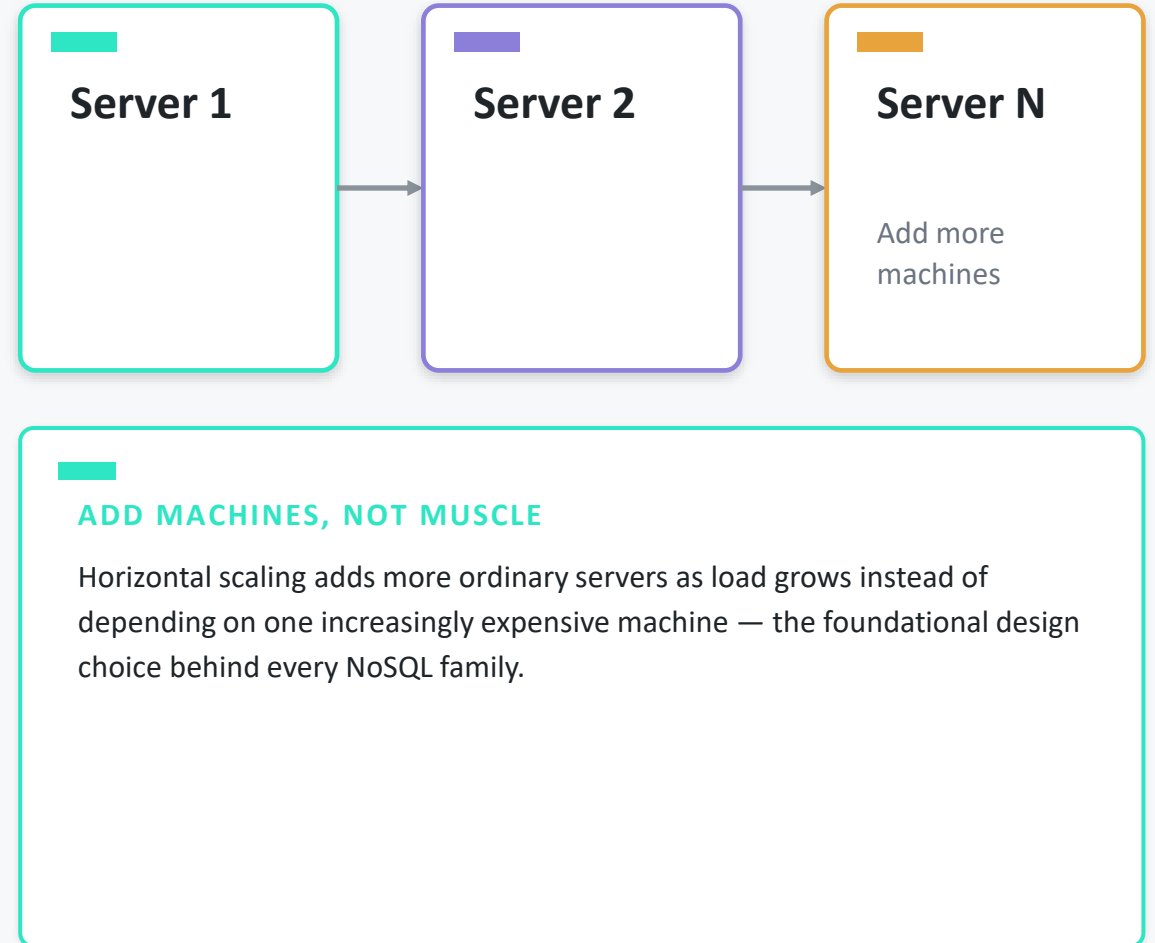
The relational model optimized for strong consistency and rich, ad-hoc querying over a schema known in advance. NoSQL systems exist for workloads where those specific guarantees matter less than raw horizontal scale and schema flexibility.

Vertical Scaling vs. Horizontal Scaling

VERTICAL SCALING — THE RELATIONAL DEFAULT



HORIZONTAL SCALING — THE NOSQL DEFAULT



Scaling One Feature: Posting Photos

● 1,000 users

One server handles photo writes in milliseconds. One table holds everything. No change needed.

● 10 million users

Peak-hour writes spike. Vertical scaling buys time, but the ceiling is close.

● 500 million users

Shard photos by user ID across many servers. Document storage avoids a migration per new post type.

■ WHY THE ARITHMETIC MATTERS

If peak writes double every year, a server with 2x headroom today is full in about 12 months. Growth rate, not size alone, forces the move to horizontal scaling. (Figures are illustrative.)

Characteristics of NoSQL Databases

● Schema-flexible

Records don't need to share fields; new fields added on the fly, no ALTER TABLE, no downtime.

● Horizontally scalable by design

Built to spread data and load across many ordinary servers, not one powerful one.

● Often eventually consistent

Many relax strict consistency for availability and speed — the BASE model instead of relational ACID.

● Optimized for specific access patterns

Each family is built around the handful of operations it needs to be extremely fast at.

Relational vs. NoSQL Databases

Dimension	Relational (SQL)	NoSQL
Schema flexibility	Fixed, defined in advance; changes need ALTER TABLE	Flexible or absent; documents/records can differ in shape
Scaling model	Primarily vertical; horizontal scaling is possible but hard	Horizontal by default (add more servers)
Consistency model	Strong consistency, ACID transactions	Often eventual consistency, BASE model (varies by product)
Query language	SQL — standardized across vendors	Varies per product (MongoDB API, CQL, Gremlin, ...)
Typical use case	Banking, ERP — strict integrity, complex joins	Content mgmt, catalogs, real-time analytics, social graphs

A SPECTRUM, NOT A STRICT LINE

Modern relational engines support JSON columns and read replicas; several NoSQL products (MongoDB included) now support multi-document ACID transactions. This table describes each family's default center of gravity, not an absolute rule.

Types of NoSQL Databases

● Document Store — MongoDB

Stores self-contained, JSON-like documents grouped into collections. Each document can have its own shape.

● Key-Value Store — DynamoDB / Redis

Stores an opaque value against a unique key — extremely fast lookups, minimal querying beyond “get this key.”

● Column-Family Store — Cassandra

Stores data in wide rows grouped by column families, tuned for very high write throughput across many nodes.

● Graph Database — Neo4j

Stores nodes and the relationships between them, purpose-built for traversal queries like “friends of friends.”

One User Profile, Four Data Models

DOCUMENT (MONGODB)

Nested, one read

{ name, city, posts: [...] } is returned in a single fetch

Flexible shape

Each user document can hold different fields

Maps directly to objects

Application objects map straight onto JSON-like documents

KEY-VALUE / COLUMN-FAMILY / GRAPH

Key-value: fast, opaque

user:4821 maps to one blob; nothing can be queried inside it

Column-family: wide rows

One row per user, with columns grouped by feature

Graph: relationships first

FOLLOWS edges are followed directly, hop by hop

NoSQL Data Models

Model	How data is organized	Example subject
Document	Self-contained, nested, JSON-like structure per record	MongoDB BSON documents (Lecture 27)
Key-value	An opaque value retrieved only by its exact key	Redis, DynamoDB
Column-family	Rows keyed by ID; columns grouped into families that vary row to row	Apache Cassandra
Graph	Data modeled explicitly as nodes and edges; traversal is the primitive operation	Neo4j

SAME IDEA, DIFFERENT SHAPE

Each model pairs its product with a distinct way of organizing data — the document model (self-contained, nested, JSON-like) is the one this course goes hands-on with, starting next lecture.

Advantages and Limitations

ADVANTAGES

Scales horizontally

Add commodity servers instead of a bigger one

Flexible schema

Iterate on the data model without migrations

Fast for its access pattern

Purpose-built for the operations it targets

LIMITATIONS

Weaker consistency guarantees

Eventual consistency can surprise developers used to ACID

No standard query language

Skills and tooling don't transfer between products like SQL does

Weaker or absent joins

Multi-collection/table relationships are harder to query in one step

Friend-of-Friend Query, Compared

● Relational (SQL)

Two self-joins on a Friendships table; cost grows with each extra hop

● Graph (Neo4j)

Traverse FOLLOWS edges directly; each hop is a pointer follow

● Document (MongoDB)

Store a friends array per user: one fetch, but large friend lists grow heavy

● Why it matters

The query shape decides the model: join-heavy or traversal-heavy

● Practical rule

Start from the most frequent read path, then pick the model

NoSQL Use Cases

● Content management

Articles, product pages, profiles that naturally vary in shape (document stores).

● Real-time analytics

High-velocity event and metrics data written far more often than updated (column-family).

● Product catalogs

Items with wildly different attribute sets — a book vs. a laptop — forcing sparse relational tables otherwise.

● Social graphs

Friend networks, recommendations, fraud detection built around “who's connected to whom” (graph).

● Session stores and caches

Simple, extremely fast get/set operations against a key (key-value).

Choosing a Database, Step by Step

USE RELATIONAL WHEN

Data is highly structured

The schema is stable and integrity rules matter most

Transactions span many rows

Money moves, orders, inventory: ACID is required

Queries are ad-hoc

Reporting and joins across many tables

USE NOSQL WHEN

Shape varies per record

Catalogs, profiles, and content with different fields

Scale exceeds one server

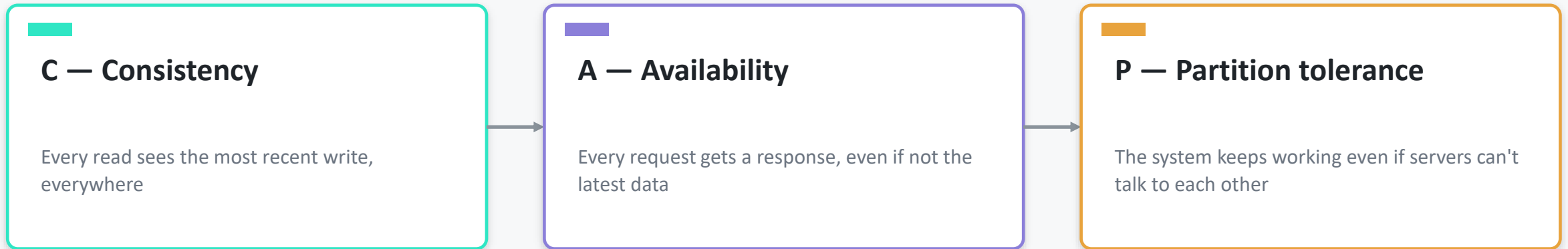
Write volume or data size outgrows vertical scaling

Access is by key or traversal

Sessions, caches, social graphs, event streams

CAP Theorem Overview

Once a database is distributed — data spread across more than one machine — it can only fully guarantee two of these three properties at once, whenever a network partition occurs:



NOT ALL-THE-TIME — ONLY DURING A PARTITION

Outside an actual network partition, a well-built distributed system can offer strong consistency and high availability simultaneously. Most real products (MongoDB included) let you tune how they behave during a partition, rather than being locked into pure CP or pure AP.

ACID vs. BASE

● Atomicity vs. Basically Available

ACID: all-or-nothing commits. BASE: the system keeps answering, even during partial failure.

● Consistency vs. Soft state

ACID: every commit leaves data valid. BASE: replicas may briefly hold inconsistent state.

● Isolation vs. Eventual consistency

ACID: concurrent transactions do not interfere. BASE: replicas converge given time.

● Durability vs. Availability first

ACID: committed work survives crashes. BASE: favors availability over immediate agreement.

CAP in Practice: The Lahore/Karachi Link Fails

Servers span two data centers, replicating every write to each other. The link between them fails — partition tolerance isn't optional, since the failure already happened. For every write arriving while the link is down:

CHOOSE CONSISTENCY (GIVE UP AVAILABILITY)



Reject the write

Karachi refuses new writes until it can confirm with Lahore again



Result: CP system

Never returns stale data, but some requests fail during the partition

CHOOSE AVAILABILITY (GIVE UP CONSISTENCY)



Accept the write locally

Karachi keeps serving requests using only its own local data



Result: AP system

Always responds, but Lahore and Karachi may briefly disagree



EVENTUAL CONSISTENCY, AND THE CA OUTLIER

Once the link returns, an AP system reconciles the divergent writes — exactly what “eventual consistency” means. A single-server relational database is usually called CA — it was never partition-tolerant, because there was only ever one server to ask.

Consistency or Availability During a Partition

CHOOSE CP WHEN

Money and inventory

A stale balance is worse than a refused request

Coordinated writes

The Lahore side must confirm before accepting a write

Strict uniqueness

Two users must never claim the same seat

CHOOSE AP WHEN

Feeds, likes, carts

Serving slightly stale data beats an error page

Read-heavy catalogs

Users prefer a fast answer over the newest answer

Reconciliation is feasible

Conflicting writes can be merged after the partition heals

Key Takeaways

- Relational databases hit real limits at web scale: vertical scaling's ceiling, expensive joins, rigid schemas.
- NoSQL trades strict schema and strong consistency for horizontal scalability and schema flexibility — a deliberate trade-off, not a strict upgrade.
- Four major families: document (MongoDB), key-value (DynamoDB, Redis), column-family (Cassandra), graph (Neo4j).
- CAP theorem: a distributed system guarantees only two of Consistency, Availability, Partition tolerance during an actual partition.
- NoSQL fits content management, analytics, flexible catalogs, and social graphs — the right tool for specific workloads, not a universal replacement.